Week 1 Lab - Build a Maze-Solving AI

Today you will build an AI agent that solves mazes. This AI does **not** learn from data yet. It solves a problem by searching through possible states.

You do not need to know data structures before starting. We will use them because the maze forces us to need them:

- a `list` for a path or trail
- a `set` for cells we already visited
- a `queue` as a waiting line for BFS
- a `stack` as a last-in-first-out pile for DFS
- a `dictionary` to remember where each cell came from

Quick Python and search vocabulary recap

Before we build the maze solver, here are the structures we will use. You do **not** need prior data-structures knowledge.

Term	Mental model	Search use today
`tuple` like `(row, col)`	one fixed coordinate	represents one maze cell
`list`	ordered trail or pile	stores a path; can act like a stack
`set`	stamp collection	remembers visited cells so we do not loop forever
`dict`	lookup table: key -> value	remembers `came_from[cell] = previous_cell`
stack	last in, first out pile	creates DFS
queue	first in, first out line	creates BFS
graph	network of choices	maze cells are nodes; legal moves are edges

Prediction question: if two cells point to each other and we do not use a `visited` set, what could happen?

```
# Quick warmup: run this cell and predict the output before you run it.
path = [(1, 1), (1, 2)]
path.append((1, 3))

visited = set()
visited.add((1, 1))
visited.add((1, 2))
```

```
came_from = {(1, 2): (1, 1), (1, 3): (1, 2)}

print("current cell:", path[-1])
print("have we visited (1, 2)?", (1, 2) in visited)
print("where did (1, 3) come from?", came_from[(1, 3)])

# TODO: In one sentence, explain why path, visited, and came_from are
three different memories.

from collections import deque

MAZE_1 = [
    "#########",
    "#S..#...#",
    "#.#.#.#G#",
    "#.#...#.#",
    "#.#####.#",
    "#.......#",
    "#########",
]

def show_maze(maze):
    for row in maze:
        print(row)

show_maze(MAZE_1)
```

```
#########
#S..#...#
#.#.#.#G#
#.#...#.#
#.#####.#
#.......#
#########
```

Part 1 - Representing the problem

A maze cell is written as `(row, col)`. The top-left cell is `(0, 0)`.

Prediction: What is the coordinate of `S` in `MAZE_1`? What is the coordinate of `G`?

```
def find_symbol(maze, symbol):
    """Return (row, col) where symbol appears in the maze."""
    # TODO 1: loop through rows and columns
    # Hint: maze[r][c] gives the character at row r, column c
    pass

start = find_symbol(MAZE_1, "S")
goal = find_symbol(MAZE_1, "G")
```

```
print("start:", start)
print("goal:", goal)

# Checkpoint: these should not be None
assert start is not None
assert goal is not None
```

Part 2 - Legal moves

From one cell, the agent can try moving up, down, left, or right. A move is legal only if:

1. it stays inside the maze
2. it does not hit a wall `#`

```
def neighbors(maze, cell):
    """Return all legal neighboring cells from the current cell."""
    row, col = cell
    possible_moves = [
        (row - 1, col),  # up
        (row + 1, col),  # down
        (row, col - 1),  # left
        (row, col + 1),  # right
    ]
    legal = []

    # TODO 2: add only legal moves to the legal list
    # Hint: check 0 <= r < len(maze) and 0 <= c < len(maze[0])
    # Hint: maze[r][c] != "#"

    return legal

print("Neighbors of start:", neighbors(MAZE_1, start))
assert len(neighbors(MAZE_1, start)) > 0
```

Part 3 - Reconstruct the path

During search, we will remember where each cell came from:

```
came_from[next_cell] = current_cell
```

This dictionary lets us walk backward from the goal to the start after the search succeeds.

```
def reconstruct_path(came_from, start, goal):
    """Return the path from start to goal using the came_from
dictionary."""
    # TODO 3: start at goal and walk backward until start
    # Hint: build the path backward, then reverse it
    pass
```

Part 4 - Breadth-first search (BFS)

BFS uses a **queue**: a first-in, first-out waiting line. This means cells discovered earlier are explored earlier.

Why this matters: in an equal-cost maze, BFS checks all paths of length 1, then length 2, then length 3, and so on. That is why it finds a shortest path.

```
def bfs(maze, start, goal):
    """Return (path, explored_count) using breadth-first search."""
    frontier = deque([start])       # waiting line
    visited = {start}               # stamp collection of discovered
cells
    came_from = {start: None}       # map: cell -> previous cell
    explored_count = 0

    while frontier:
        current = frontier.popleft()  # remove from front of queue
        explored_count += 1

        # TODO 4: if current is the goal, reconstruct and return the
path

        # TODO 5: loop through neighbors of current
        # If a neighbor has not been visited:
        # - add it to visited
        # - remember where it came from
        # - add it to the frontier

    return None, explored_count

path, explored = bfs(MAZE_1, start, goal)
print("Path:", path)
print("Path length:", None if path is None else len(path) - 1)
print("Explored cells:", explored)
assert path is not None
assert path[0] == start and path[-1] == goal
```

Part 5 - Show the solution path

```
def draw_path(maze, path):
    """Print the maze with the solution path marked using *."""
    if path is None:
        print("No path found.")
        return
    path_cells = set(path)
    for r, row in enumerate(maze):
        line = ""
        for c, ch in enumerate(row):
            if (r, c) in path_cells and ch not in "SG":
```

```
                line += "*"
            else:
                line += ch
        print(line)

draw_path(MAZE_1, path)
```

Part 6 - Depth-first search (DFS)

DFS uses a **stack**: the newest cell added is the first one explored. It can get lucky, but it does not guarantee a shortest path.

Prediction before coding: Will DFS explore more or fewer cells than BFS on this maze? Will its path be shorter, longer, or the same?

```
def dfs(maze, start, goal):
    """Return (path, explored_count) using depth-first search."""
    frontier = [start]              # stack
    visited = {start}
    came_from = {start: None}
    explored_count = 0

    while frontier:
        current = frontier.pop()    # remove from top of stack
        explored_count += 1

        # TODO 6: complete DFS by adapting your BFS code

    return None, explored_count

path_dfs, explored_dfs = dfs(MAZE_1, start, goal)
print("DFS path length:", None if path_dfs is None else len(path_dfs)
- 1)
print("DFS explored cells:", explored_dfs)
draw_path(MAZE_1, path_dfs)
```

Part 7 - Heuristic idea

A heuristic is an estimate. For a grid, one useful estimate is Manhattan distance:

`abs(row1 - row2) + abs(col1 - col2)`

It ignores walls, so it is not perfect. But it can still guide search.

```
def manhattan(a, b):
    """Return Manhattan distance between cells a and b."""
    # TODO 7: implement the formula
    pass
```

```
print(manhattan((2, 3), (7, 11)))  # expected 13
assert manhattan((2, 3), (7, 11)) == 13
```

Reflection

Answer in a markdown cell:

1. Is your maze solver learning? Why or why not?
2. Which data structure helped avoid loops?
3. Why does BFS find a shortest path here?
4. Describe one maze where DFS might perform badly.